Data Engineering 2025/2026
Assignment 2 (Group): City Live Monitor Data Platform

The main purpose of this assignment is to build an end-to-end, hybrid data platform that ingests, transforms, models, and serves both batch and real-time streaming data for analytics.

Each group will construct a "City Live Monitor" for one or more cities, combining historical context with live updates. You will architect a complete data pipeline from raw data extraction to interactive dashboards.

1 Objectives

1. Design a modular and scalable data pipeline that supports both *batch* and *streaming* ingestion.
2. Integrate multiple heterogeneous data sources using open data APIs and/or datasets.
3. Implement data transformation workflows using modern data tools and frameworks.
4. Build data models (both analytical and operational) to support downstream use cases.
5. Develop and deploy interactive dashboards or visualizations summarizing key insights.
6. Demonstrate key engineering practices such as reproducibility, monitoring, reliability, and version control.

2 Data Sources

You must integrate at least three distinct data sources, of which at least one is batch and one is truly real-time (event-driven). Use publicly available APIs or datasets. A shared data lake or local workspace can serve as a common data zone.

1. **Batch/Static Source**
   Use the NYC TLC Trip Record Data (Yellow/Green Cab datasets in Parquet or CSV format), historical weather records, census or demographic datasets.

2. **Real-time Periodic Source**
   Air quality or environmental monitoring (e.g., openaq.org), weather sensors, or live traffic conditions from municipal open data platforms.

3. **Real-time/Event-based Source**
   A public streaming or event-driven API such as:

- Public transit telemetry (GTFS Realtime feeds)
- City bike availability (e.g., CitiBike live feeds)
- Social media event data (e.g. Twitter stream on geotagged locations)
- IoT traffic cameras or 511NYC live metadata (https://511ny.org/)

You may optionally include custom or simulated streaming data to enrich your use case.

3 Ingestion Layer

The goal is to understand the data characteristics, establish quality baselines, and set up a reproducible development environment for both batch and streaming ingestion.

3.1 Batch Data Ingestion

- Write code to programmatically download multiple months of NYC Taxi data
- Implement incremental download logic (avoid re-downloading existing files)
- Support parallel downloads for multiple months
- Include retry mechanisms with exponential backoff for failed downloads
- Validate downloaded files (check file size, format, and basic schema)

3.2 Streaming Data Ingestion

- Poll chosen public APIs on a scheduled interval (30-60 seconds recommended)
- Emit events to a message bus (Apache Kafka)
- Implement exactly-once semantics where possible
- Handle API failures gracefully (circuit breaker pattern)

3.3 Data Profiling

- Profile ALL ingested data sources (batch and streaming samples)
- Generate comprehensive data quality report
- Identify and document data issues

3.4 Infrastructure Setup

- Set up complete development environment
- All services containerized with Docker Compose
- Version control with Git
- Environment-based configuration

Recomended Infrastructure/Services:

- Apache Kafka (for streaming data)
  - Kafka Connect (optional, for advanced integrations)
- LakeHouse, e.g., Delta Lake, MinIO (for Gold layer)

- Apache Airflow (for orchestration)
- Apache Superset (for visualization)
- Prometheus + Grafana (optional, for monitoring)

3.5 Deliverables

1. Data Profiling Report (PDF or Markdown)

- Executive summary of data quality
- Schema documentation for all sources
- Statistical profiles with visualizations
- List of identified data quality issues with severity ratings
- Recommendations for data cleaning strategies

2. Working Development Environment (example of README.md sections)

```
# Project name
## Prerequisites
## Quick Start
## Services
## Architecture
[Diagram or link to architecture.md]
```

3. Code Repository

- Commits with meaningful messages
- `.gitignore` excluding data files and credentials
- Branching strategy documented (main + dev branches)
- Each team member with visible contributions

4 Stream & Batch Processing Layer

Transform raw data into clean, normalized, and analytics-ready datasets using appropriate processing frameworks for batch and streaming workloads.

4.1 Batch Processing (NYC Taxi Data)

- Clean and standardize 6+ months of taxi trip data
- Implement data quality checks and filtering
- Enrich with dimensional attributes
- Partition output by date for efficient querying

Proposed Transformations:

- **Data Type Casting & Standardization**

```
# Type conversions
```

```
Timestamps: Convert to proper datetime objects (UTC)
Coordinates: Cast to DECIMAL(9,6) for lat/lon
Money: Cast to DECIMAL(10,2) for all monetary fields
IDs: Standardize as STRING or INTEGER consistently
Categoricals: Map to standardized codes

# Standardization rules
Borough names: Uppercase, consistent spelling
Zone names: Remove extra spaces, standardize format
Payment types: Map to standard codes (1=Credit, 2=Cash, etc.)
```

- **Data Quality Filtering**

```
# Invalid records to REMOVE
trip_distance <= 0 OR trip_distance > 100 miles
fare_amount < 0 OR fare_amount > 500
passenger_count = 0 OR passenger_count > 6
tip_amount < 0 OR tip_amount > fare_amount
total_amount < fare_amount
pickup_datetime >= dropoff_datetime
trip_duration < 60 seconds OR > 24 hours
pickup/dropoff coordinates outside NYC bounds:
      lat: 40.5 to 40.9
      lon: -74.3 to -73.7

# Records to FLAG (keep but mark):
tip_percent > 50%
speed > 80 mph (calculated from distance/duration)
unusual pickup/dropoff times (midnight-4am on weekdays)
```

- **Feature Engineering**

```
# Required derived fields:
trip_duration_minutes: (dropoff - pickup) in minutes
trip_speed_mph: (distance / duration) * 60
hour_of_day: EXTRACT(HOUR FROM pickup_datetime)
day_of_week: EXTRACT(DOW FROM pickup_datetime)
is_weekend: BOOLEAN (Saturday/Sunday)
is_rush_hour: BOOLEAN (7-9am, 5-7pm on weekdays)
tip_percentage: (tip_amount / fare_amount) * 100
cost_per_mile: total_amount / trip_distance
```

- **Outlier Detection & Handling**

```
# Use IQR method or Z-score
Flag trips with distance > 99th percentile
Flag trips with fare > 99th percentile
Cap trip_duration at 95th percentile
Document outlier handling strategy in README
```

4.2 Stream Processing (Real-Time Data)

- Parse and normalize incoming events from Kafka
- Implement stateful stream processing
- Handle late/out-of-order events
- Produce aggregated metrics

Proposed Transformations:

- **Event Envelope Parsing**

```
# Extract and validate envelope
Parse metadata (timestamp, source, event_id)
Validate payload schema
Reject malformed events to dead letter queue
Enrich with processing metadata
```

- **Watermarking**

```
# Purpose: Handle late-arriving events
Define watermark strategy (e.g., 5-minute delay)
Configure allowed lateness threshold
Document watermark logic in code comments
```

- **Windowed Aggregations**

```
# Purpose: Handle counts, averages, .. per subject, time
```

- **Late Event Handling**

```
# Purpose: Implement a startegy to handle “late” events
```

- **State-Keyed Deduplication**

```
# Purpose: Remove duplicate events within window
```

- **Out-of-Order Event Detection**

```
# Track and report events arriving out-of-order (timestamp < previous)
```

4.3 Processing Orchestration

- Use **Apache Airflow** to orchestrate batch processing
- Schedule batch jobs appropriately
- Monitor processing health

4.4 Deliverables

Processing Report (Markdown or PDF)

```
# Data Processing Report (example)
```

```
## Batch Processing Summary
Records ingested: 24.5M
Records filtered: 1.2M (4.9%)
Processing time: 18 minutes
Output size: 22GB (Silver layer)

## Filtering Breakdown
| Rule          | Records Removed |     % |
|---------------|-----------------|-------|
| Negative fare |          15,432 | 0.06% |
| Zero distance |         892,341 | 3.64% |
| Invalid coord |         125,678 | 0.51% |
...

## Stream Processing Summary
Average throughput: 45 events/second
Average latency: 1.2 seconds
Late events: 2.3%
Duplicate events: 0.8%
Watermark delay: 5 minutes

## Windowed Aggregations
[Sample output tables]

## Out-of-Order Analysis
[Graph showing event arrival patterns]
```

5 Storage (The Medallion Architecture)

Implement a three-tier medallion architecture (Bronze → Silver → Gold).

5.1 Bronze Layer

Preserve raw data exactly as ingested, immutable and append-only.

- Data Retention
  - Batch: Retain all historical data (no expiration)
  - Streaming: Retain for *n* days
- Validation Requirements:
  - Row counts match source data (± deduplication)
  - No schema drift between partitions
  - All partitions have _ingestion_timestamp

5.2 Silver Layer

Produce cleaned data, with new and removed columns, data transformations, some processing, ready for analysis but not aggregated.

5.3 Gold Layer

Design an analytical data model.

- For the batch data, create a dimensional model (star schema) loaded into the lakehouse (Delta Lake, MinIO, ..)
- For the streaming data, provide aggregated metrics tables (e.g., 5-minute rolling windows).

5.4 Deliverables

The report, in PDF or Markdown, must include:

- Medallion Architecture
- Lakehouse implementation
- Data Dictionary
- Data Consistency in the 3 layers
- Query Performance Testing

6 Serving & Visualization

The goal is to build interactive dashboards using Apache Superset to serve insights from both batch and streaming data, demonstrating correlation between historical trends and real-time events.

6.1 Main tasks

- Serve analytics through Apache Superset dashboards.
- Query the combined outputs to show trends, such as comparing historical taxi volumes with real-time air quality or other events.

6.2 Requirements

- You must create at least three comprehensive dashboards, each including a minimum of 4 charts in each.

6.3 Deliverables

- Description and screenshots of each dashboard and charts.

7 Deliverables

1. Code Repository: A GitHub repository with modular, documented, and runnable code, including a README.md with clear setup instructions.
2. Architecture & Contracts: A 1-page architecture diagram and data contracts for each real-time source .
3. Populated Storage: Demonstrable Bronze, Silver, and Gold tables (including the star schema) .
4. Dashboards: Working Apache Superset dashboards combining your batch and streaming metrics.
5. Project Report: Detailing your design decisions, trade-offs, failure modes, and limitations. It can be a final combination of each phase deliverables.
6. Final Presentation: A presentation demonstrating the complete, orchestrated system and discussing the challenges encountered.